Hochschule Konstanz
Fakultät Informatik

Software Engineering

03 - Agile Development

Prof. Dr. Marko Boger

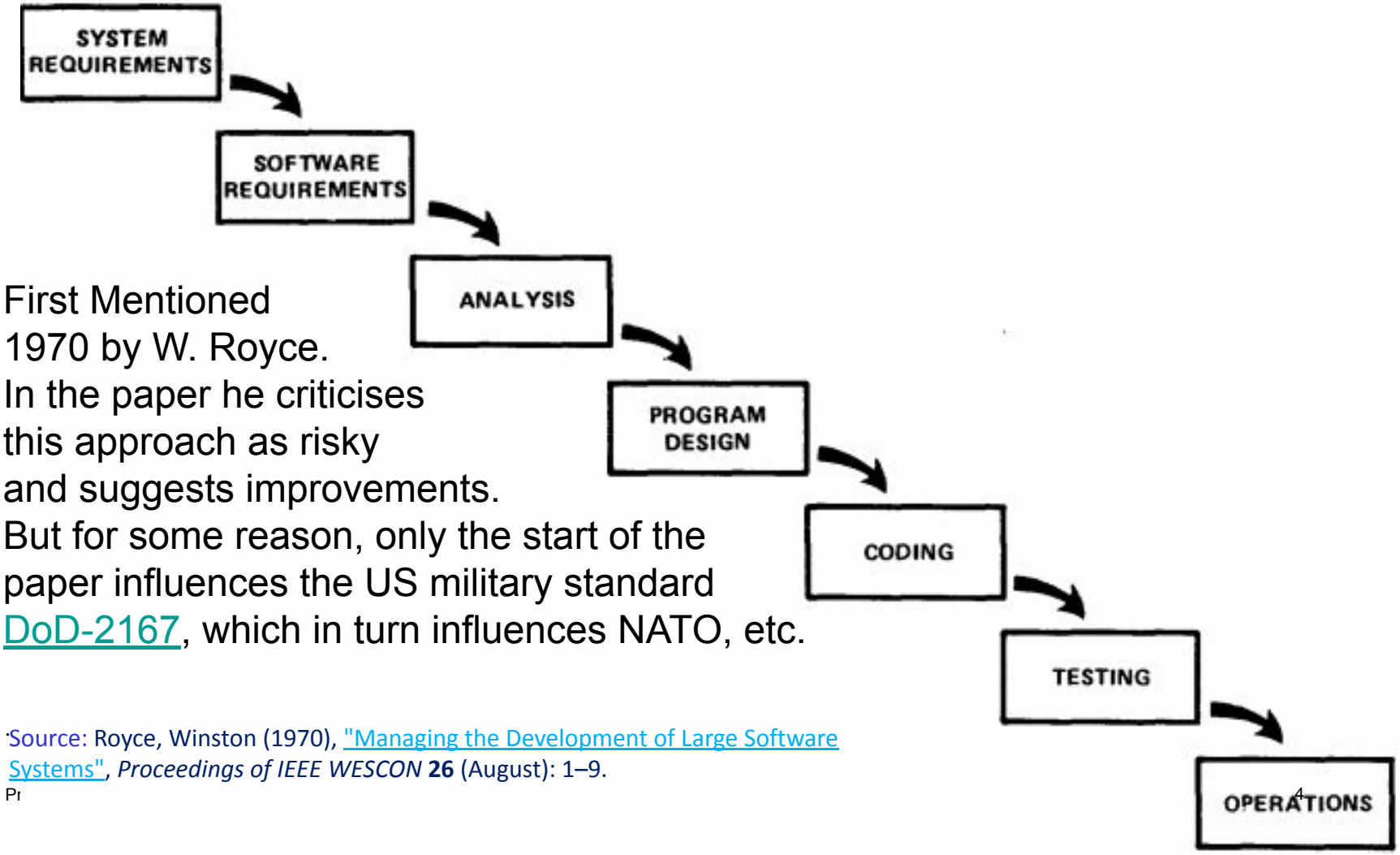
Goals of this Lecture

- Get to know several generations of software development processes
- Distinguish between heavy weight and agile processes
- Learn some lessons from XP
- Understand the core of Scrum
- Understand Test Driven Development and Behaviour Driven Development (TDD / BDD)
- Write Unit Tests and testable code

Software Development Processes

- Waterfall-Model
- Spiral-Model
- V-Model
- Rational Unified Process (RUP)
- XP
- Scrum

Waterfall Model



- First Mentioned 1970 by W. Royce.
- In the paper he criticises this approach as risky and suggests improvements.
- But for some reason, only the start of the paper influences the US military standard DoD-2167, which in turn influences NATO, etc.

Source: Royce, Winston (1970), "Managing the Development of Large Software Systems", *Proceedings of IEEE WESCON* **26** (August): 1–9.

Problem with the Waterfall Model

The main problem are unaccounted long feedback loops

Findings in late stages lead to large costs

And iterations are never confined

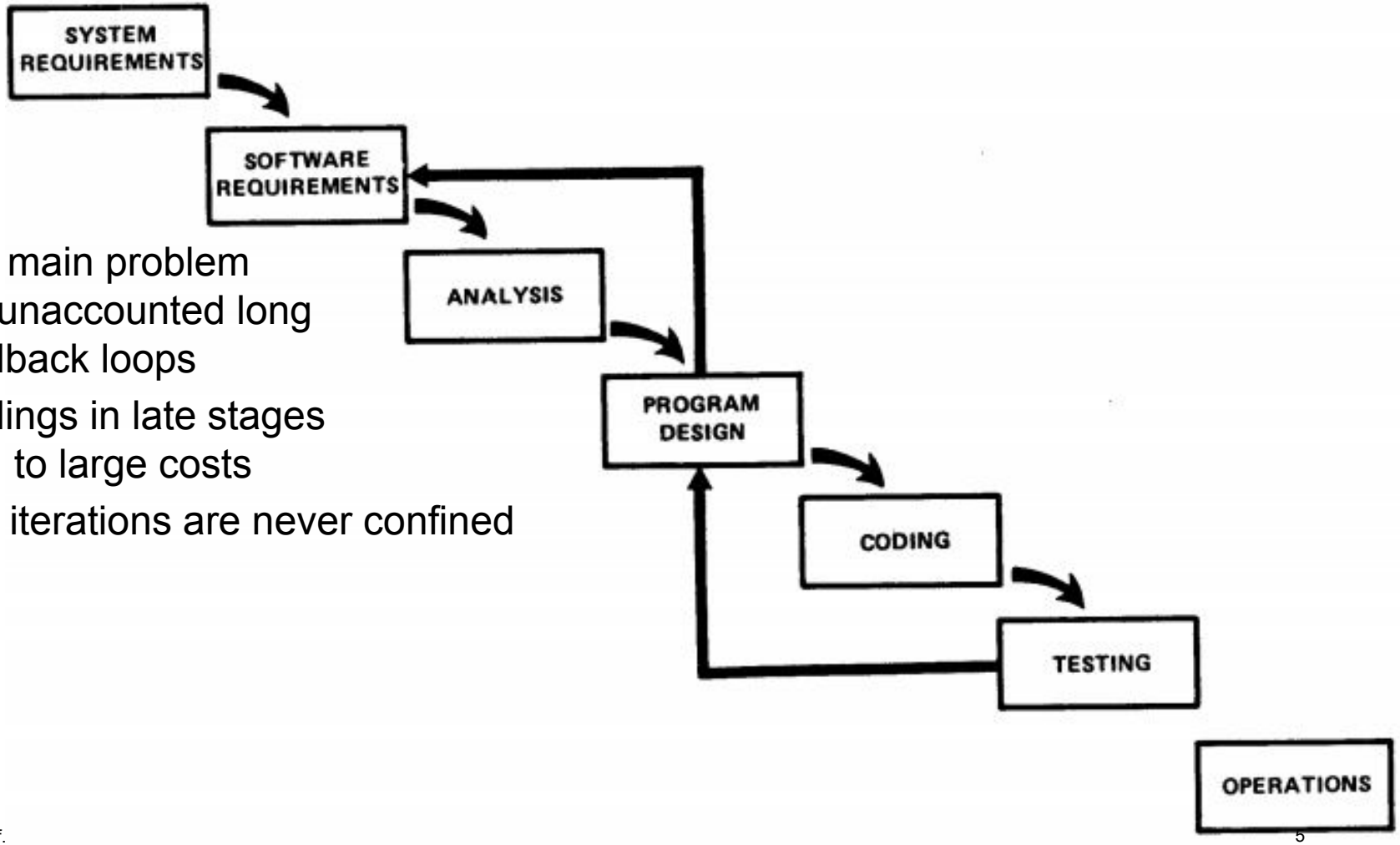


Figure 4. Unfortunately, for the process illustrated, the design iterations are never confined to the successive steps.

Video Assignment

Real Software Engineering by Glenn Vanderburg

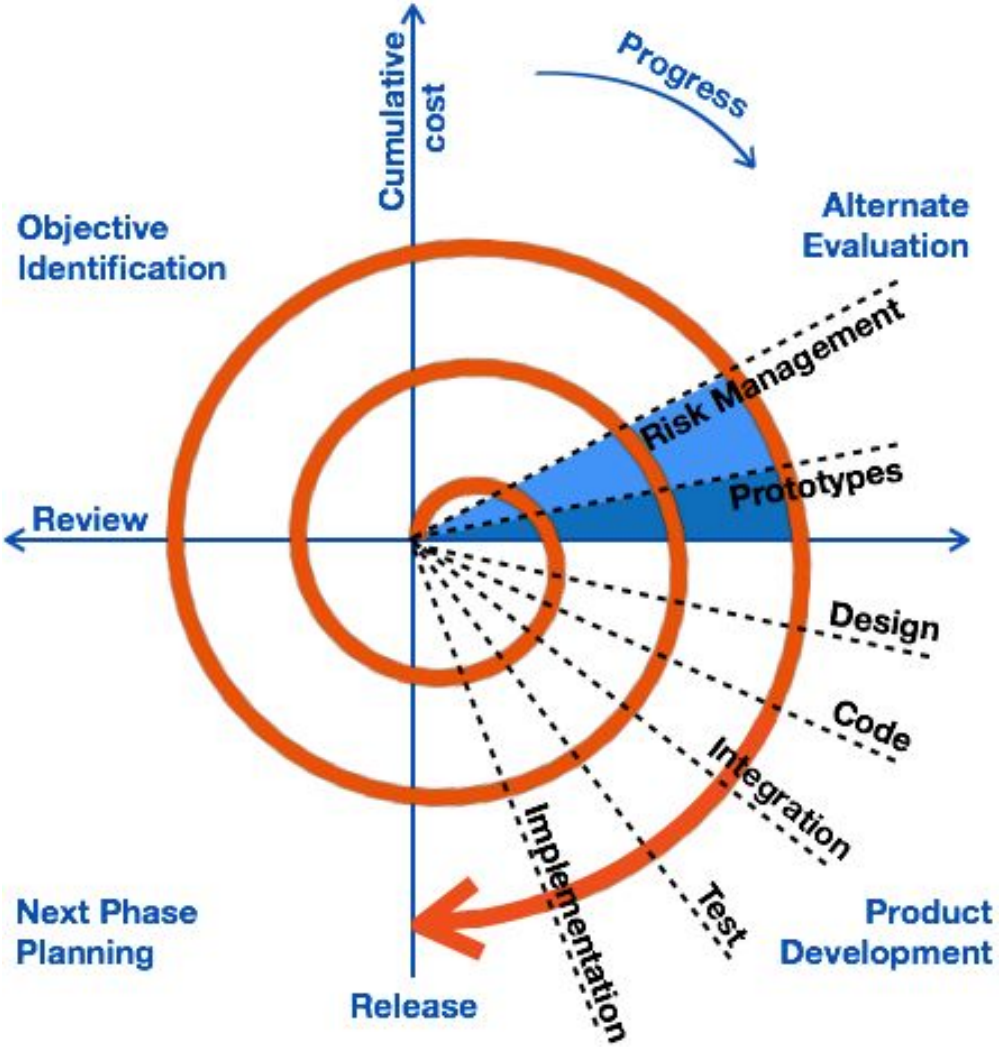
- http://www.youtube.com/watch?v=NP9AIUT9nos&feature=related

Spiral Model

Developed by
Barry Böhme
in 1985

It is the first to stress the importance of iterations

Main goal was to reduce risk as early as possible



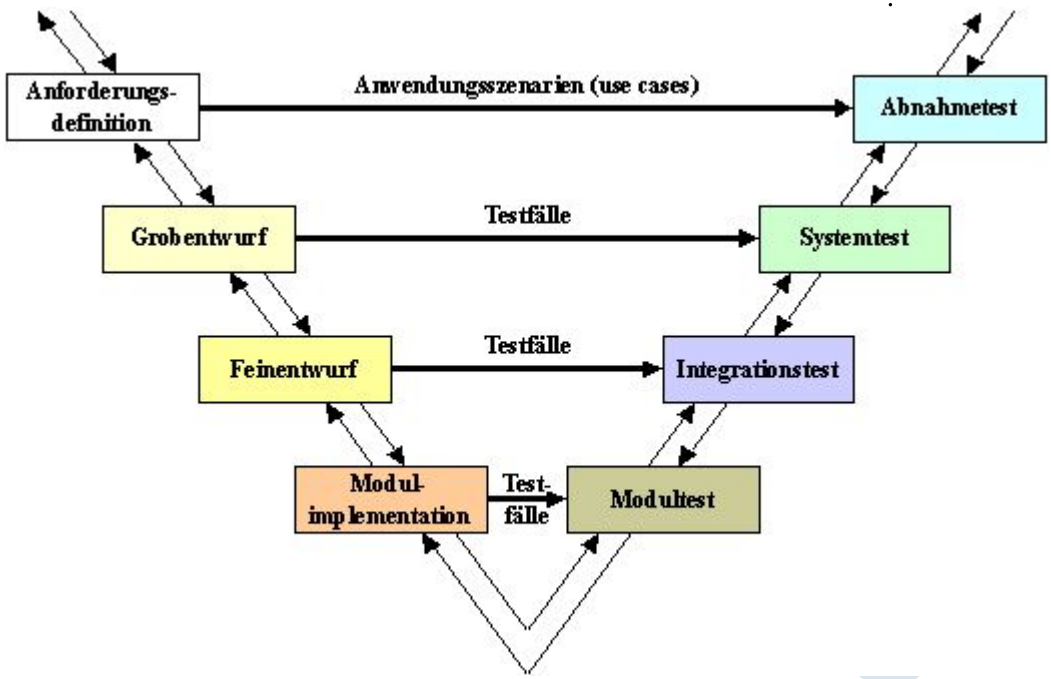
The V-Model

Developed 1992 for projects of the German Military

The left side of the "V" represents the decomposition of requirements, and creation of system specifications.

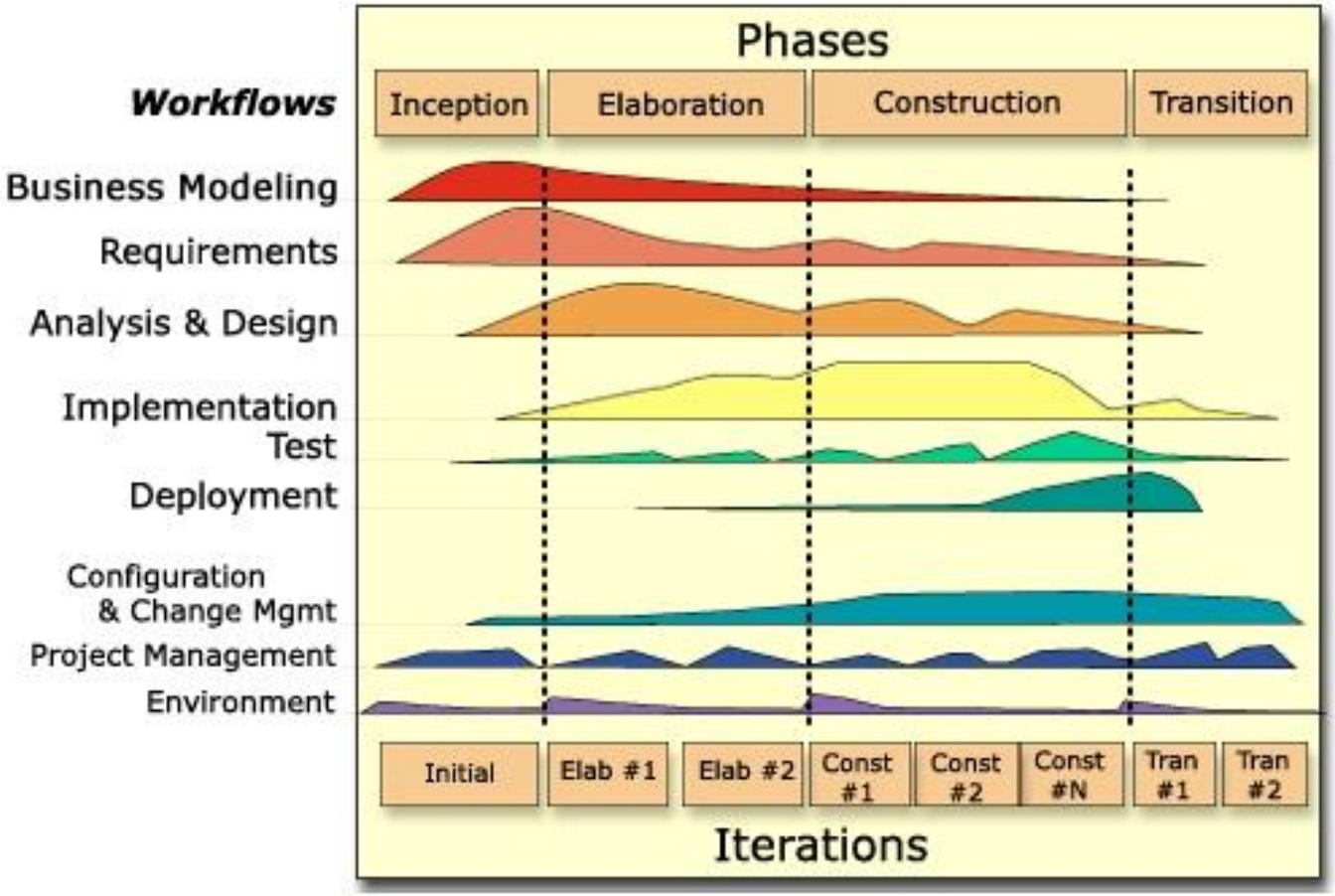
The right side of the V represents integration of parts and their validation.

Current Version:
V-Model XT



Rational Unified Process

- Developed in 1996 by Rational, now a unit of IBM
- Tooling (IBM Rational Method Composer) allows to customize it



Heavy Weight Processes

- Assumptions for heavy weight
  - Fixed Requirements
  - Construction in parts and assembly late in the process
  - A running system only after conclusion
  - Reduction of cost by building models
- The processes shown so far get under severe critique by the end of the 1990's.
- They are referred to as heavy weight, pointing to the multitude of documents to be developed.

Source: SFO Oakland Bay Bridge http://mceer.buffalo.edu/publications/bulletin/07/21-02/01_FHWA.asp

How SE differs from other Engineering Fields

- Our building material (bits or code) is essentially free and extremely flexible
- Changing and extending is possible late in the process
  - Change is frequently expected late in the process
- Creating a new instance (copy) is free
- Testing is possible instantaneously and at almost no cost
- All costs are directly related to human work
  - At a very high level of education
  - With highly creative, non-repetitive tasks

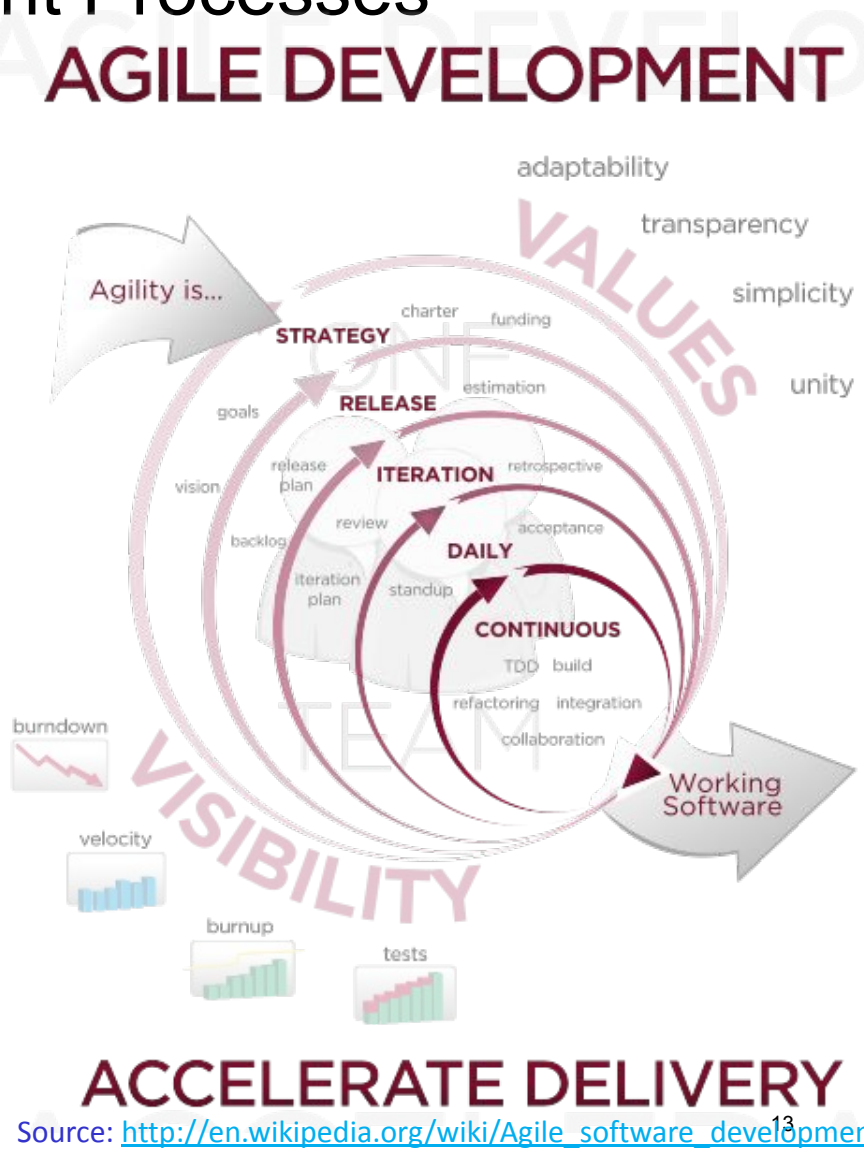
Choosing a different Metaphor

- Software as a living System
  - Should always run
  - Has an observable behaviour and can be tested
  - Is constantly extended at many points concurrently
  - Can be educated or developed
  - Has bugs

- But it is only a metaphor
  - i.e. testing does not destroy the system

Agile Development Processes

- Cristal Clear
  - 1992-97, Alister Cockburn
- Scrum
  - 1995-2001,
    Jeff Sutherland,
    Ken Schwaber
- Feature Driven Development
  - 1997, Jeff De Luca
- Extreme Programming
  - 1999, Kent Beck



Source: http://en.wikipedia.org/wiki/Agile_software_development

The Agile Manifesto

Developed at a „Think Tank“ meeting in 2001

Describes the core ideas of Agile Methods

Source:http://agilescout.com/history-of-agile-the-influencers-and-drivers-of-the-agile-movement/

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

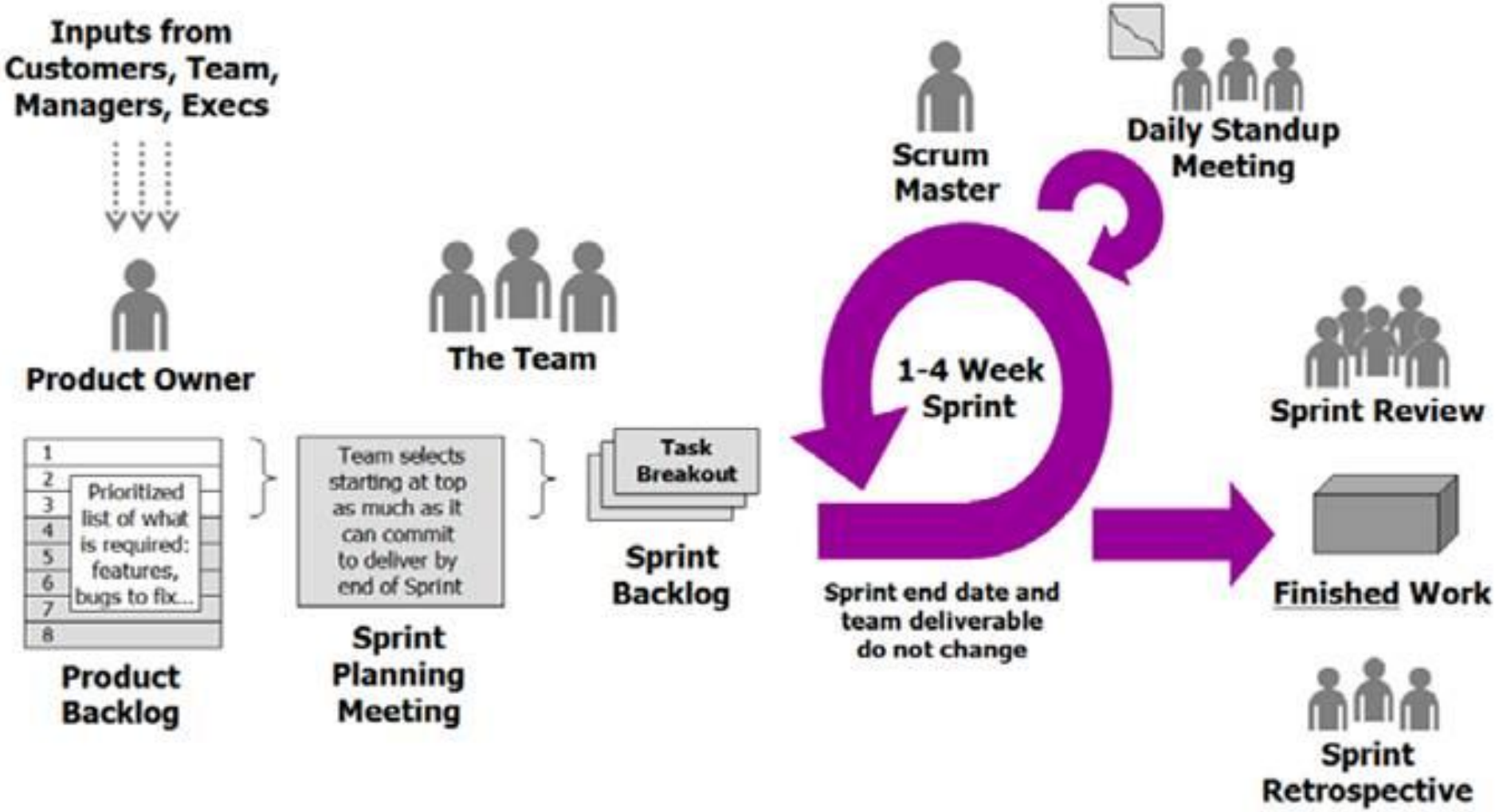
- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Principles behind the Agile Manifesto

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity--the art of maximizing the amount of work not done--is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Scrum in a Nutshell



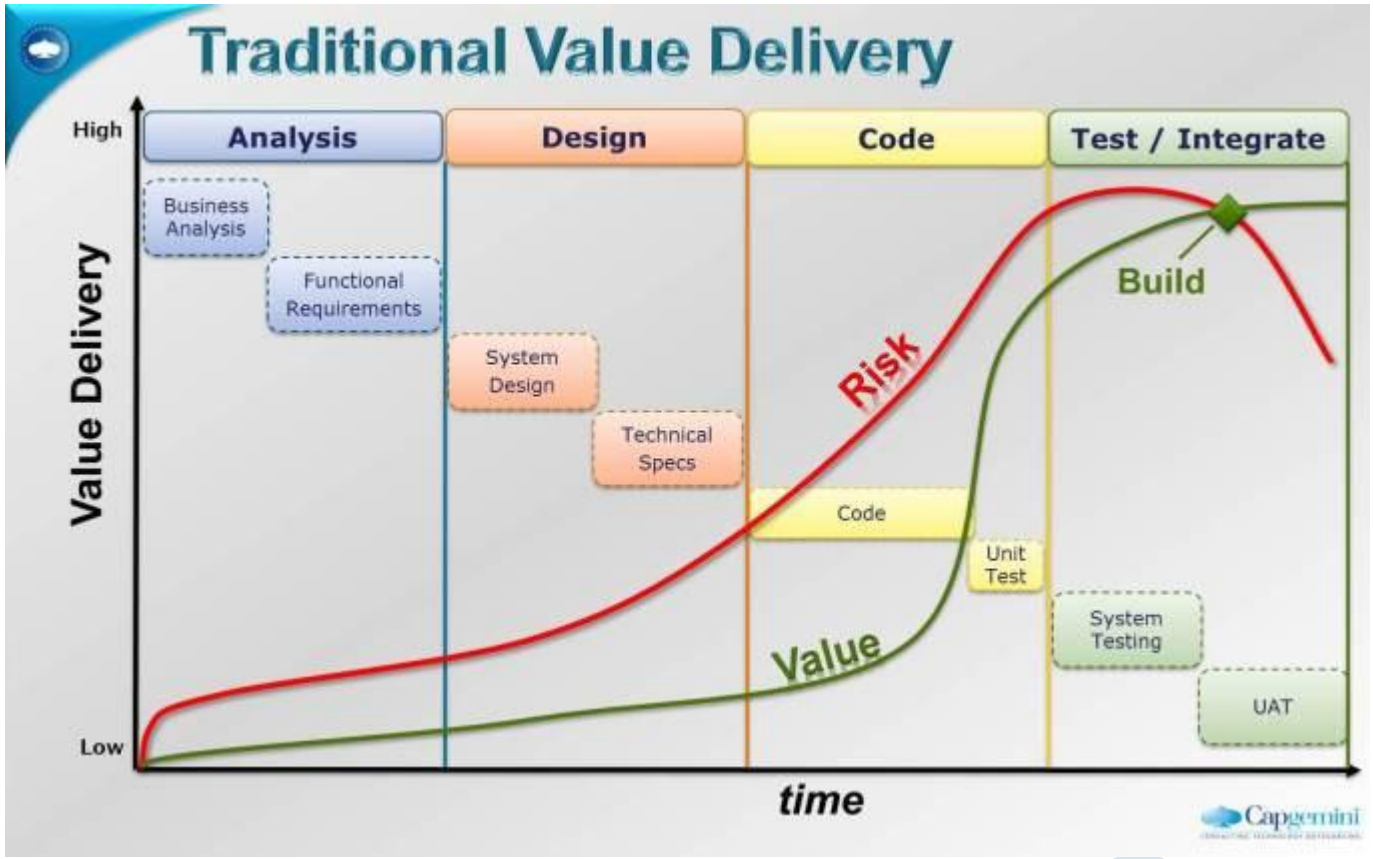
Source: http://www.sadhanbiswas.com/myblog/techsavvy/agile-methodology/

Video Assignment

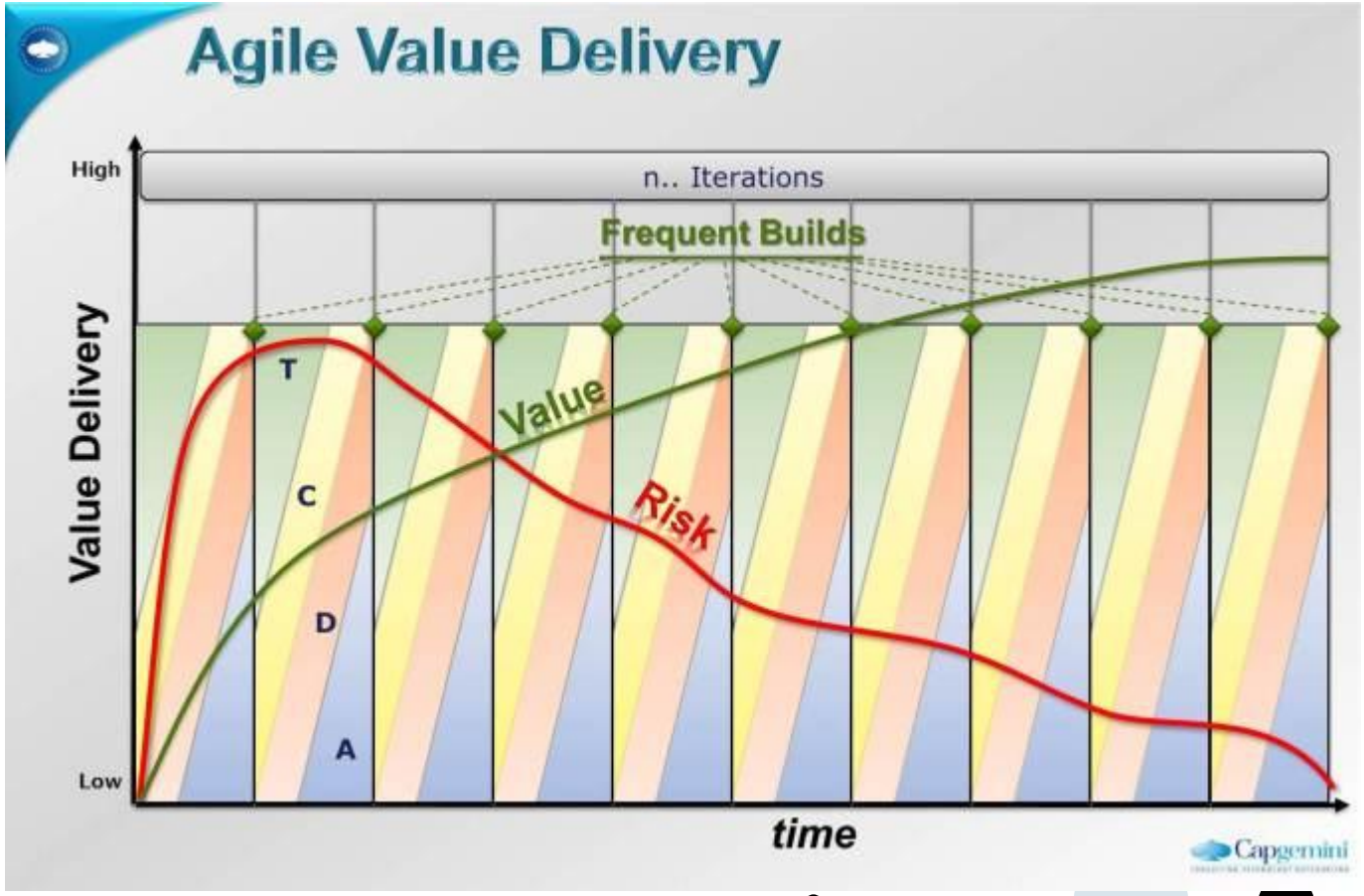
Scrum in under 10 Minutes

- http://www.youtube.com/watch?v=XU0llRltyFM

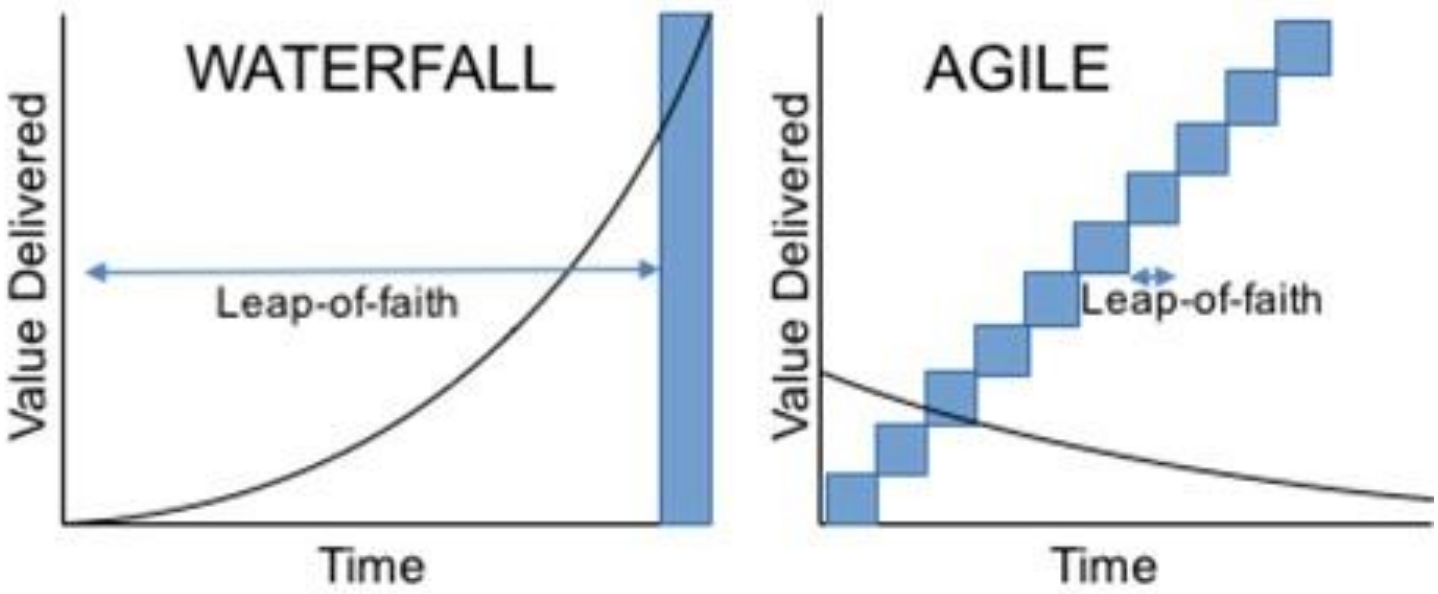
Value Delivered: Waterfall



Value Delivered: Agile



Leap of Faith



Tools to support Scrum

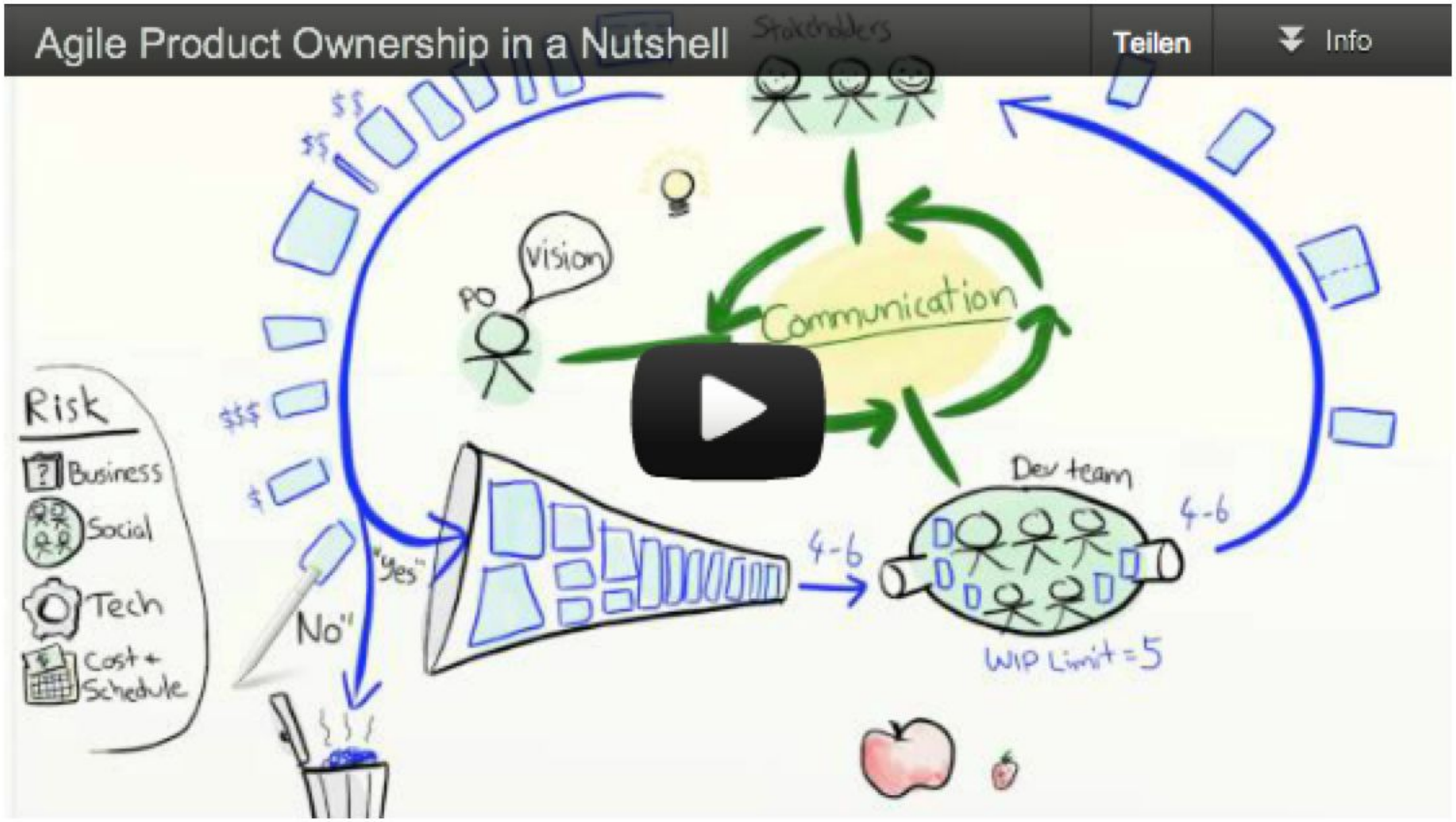
- Whiteboard
- Google Spread Sheet
- PivotalTracker
  - Webtool, almost no setup or config, free for Universities
- Ontime
  - Powerfull webtool, great Scrum support, github integration
- Jazz / Rational Team Concert
  - Full integration in Eclipse, Combines SCM and PM
- Scrumwise
  - Excellent web-based tool for Scrum
- Waffle.io
  - Based on issues on Github
- Github Issues

Limitations of Scrum

- Scrum is a Project Management Methodology
  - It helps us organize what to work on next and how to organize as a group
  - But it does not help us with how we are actually going to work with the code
  - Therefore we will look at another agile method that goes deeper
    - Extreme Programming
- Scrum is good for inhouse software development
- It is also good for software development as a service
  - If the customer really understands modern software development
- It is not very good for fixed price projects.
  - But then again, no method really is.

Product Owner in Scrum

http://blog.crisp.se/2012/10/25/henrikkniberg/agile-product-ownership-in-a-nutshell



What is Extreme Programming

- Is this Extreme Programming?

- http://www.youtube.com/watch?v=Ytu1Hxzr_Bs&feature=youtube_gdata_player

Hochschule Konstanz
Fakultät Informatik

What is Extreme Programming?

Best programming practices to 10

Prof. Dr. Marko Boger

Why extreme?

- Good Practices
  - Code reviews
  - Testing
  - Design
  - Simplicity
  - Integration
  - short iterations (weeks)
  - feedback

- taken to extremes
  - pair programming
  - unit/function testing, test-first
  - refactoring
  - simplest thing that could possibly work
  - continuous integration, daily build
  - short iterations (minutes, hours, days)
  - continuous feedback, customer on site

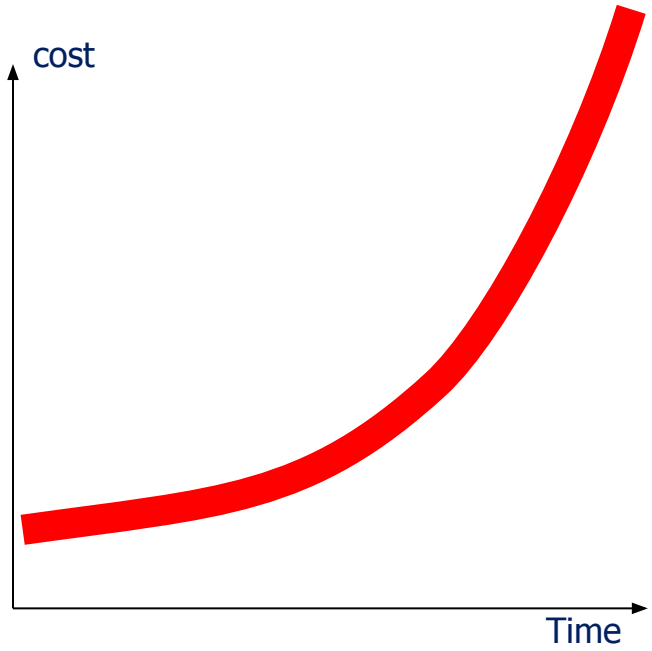
Extreme Programming

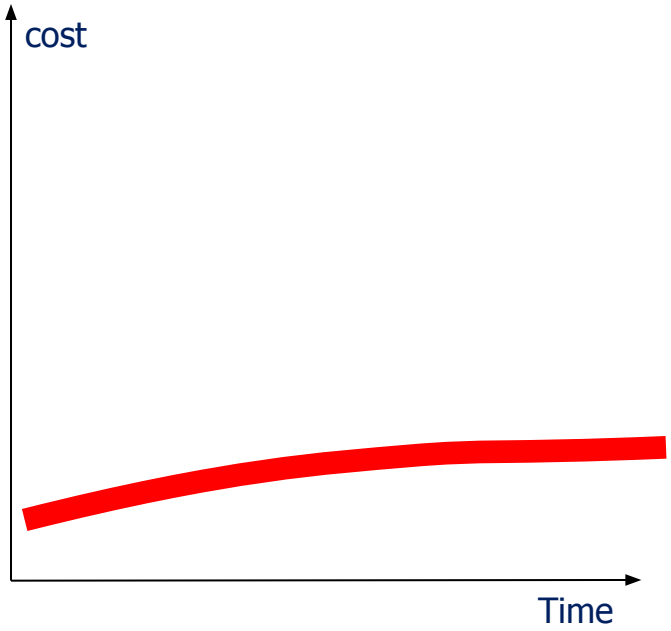
Developed by Kent Beck

- C3 Project at Chrysler:
  Salary payment system
- 1998 first Articles
- 1999 Book „Extreme Programming explained“ published
- 2000 first XP-Conference

Why does XP work?

- The “Cost of Change” Curve remains flat



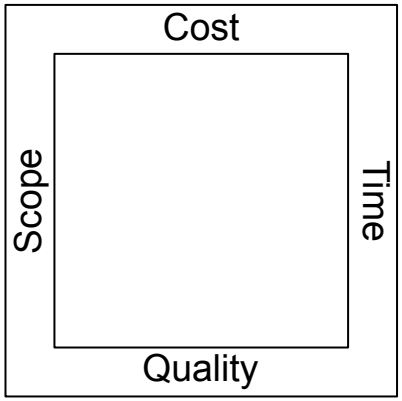


How to keep the curve flat?

- Design to change
  - Pattern, Encapsulation, Information hiding
- Automated testing of all artefacts
  - insure that changes don't break anything
  - traceability of problems
- Automated Consistency of all artefacts
  - integration of changes
  - rollback
- Simplicity
  - Simplest thing that could possibly work
  - readability
  - clear design

Parameters of a Project

- Four Variables
  - Cost
    - Budget, Team size, Resources
  - Time
    - Schedule, time of delivery
  - Quality
    - Stability, Correctness, Design, Documentation
  - Scope
    - Features, Requirements



XP Philosophy

XP has

- 4 Values
- 5 Principles
- 12 Practices

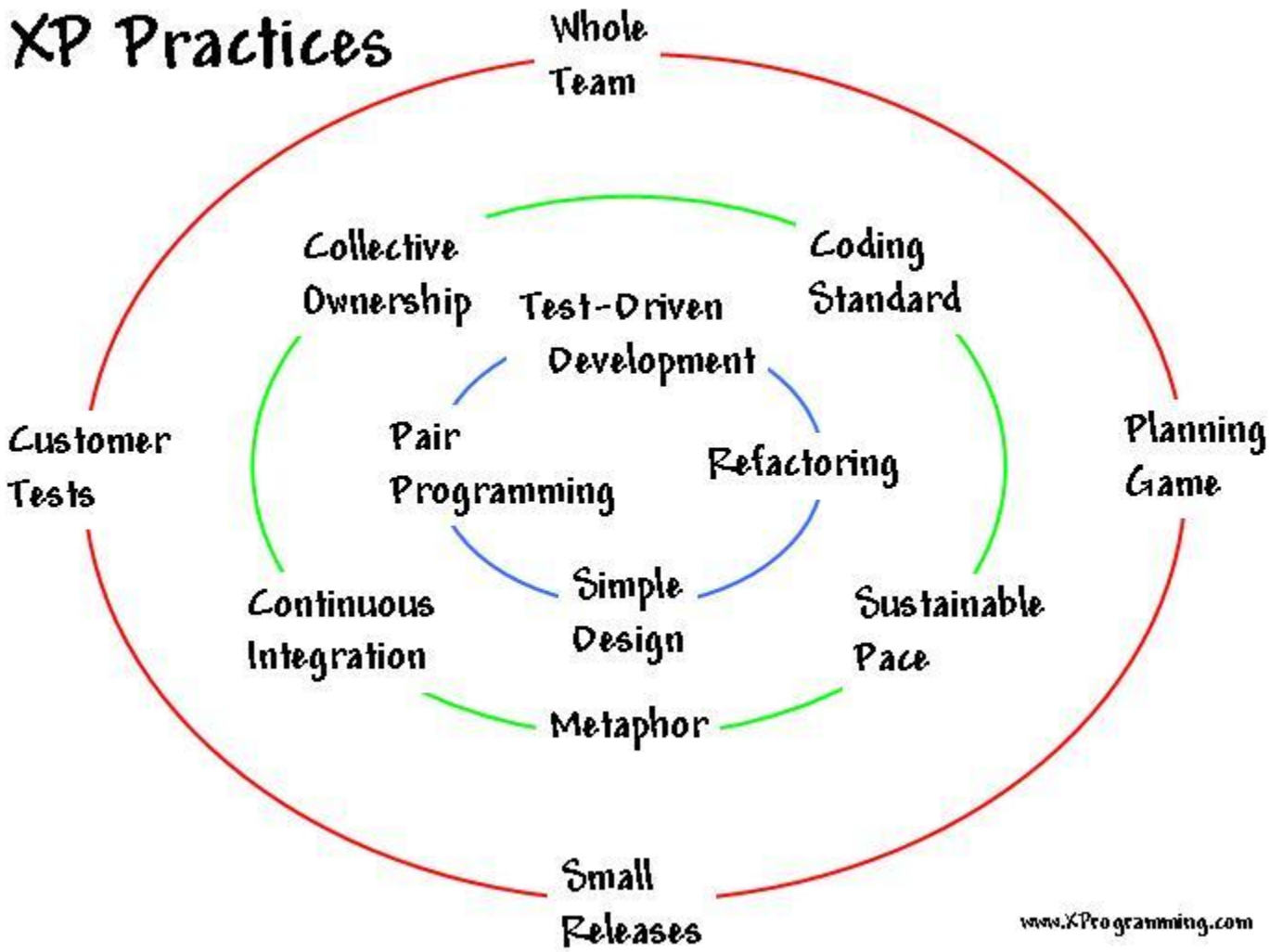
Values

- Communication
- Simplicity
- Feedback
- Courage

5 Principles

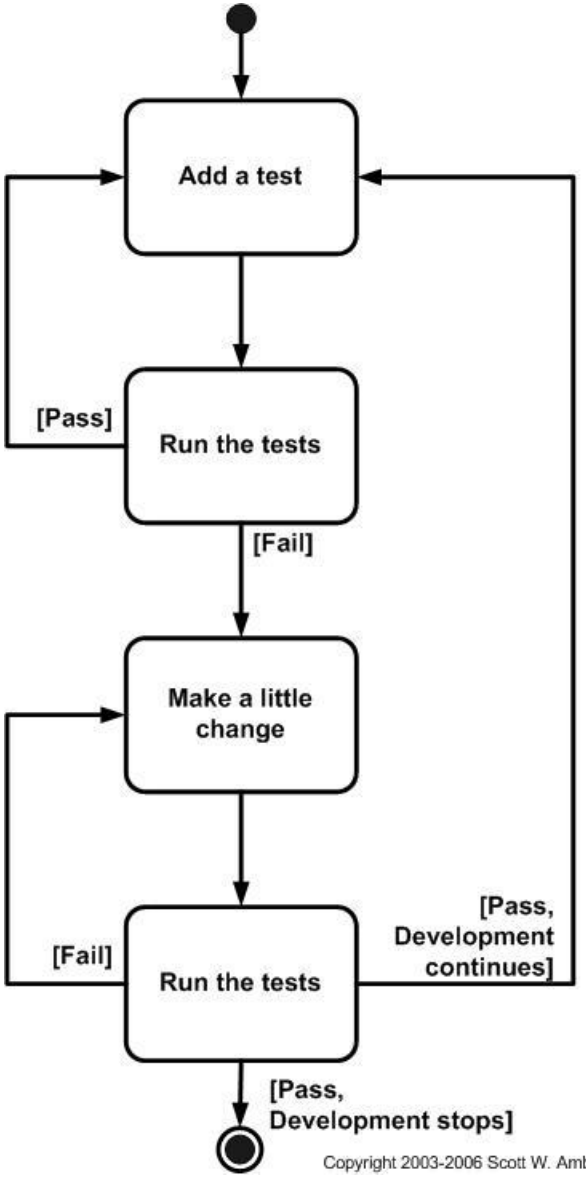
- Fundamental Principles:
  - Rapid Feedback
  - Assume Simplicity
  - Incremental Change
  - Embracing Change
  - Quality Work

12 Practices



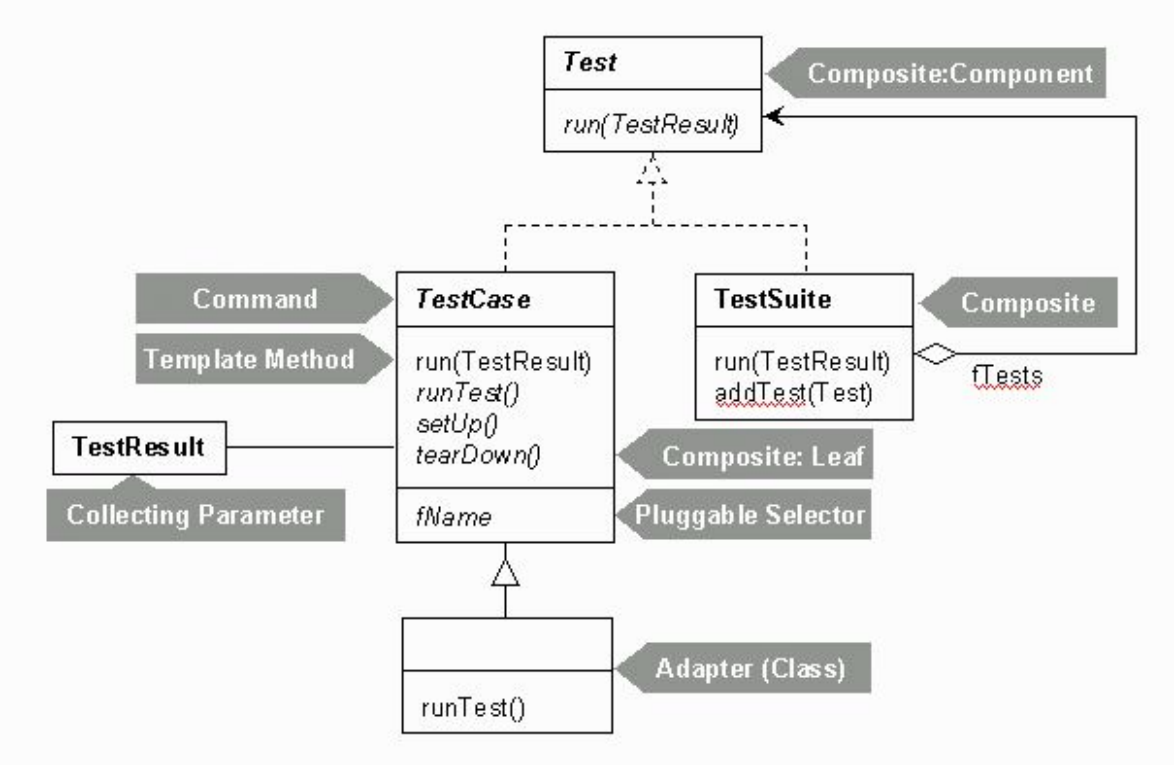
Test Driven Development (TDD)

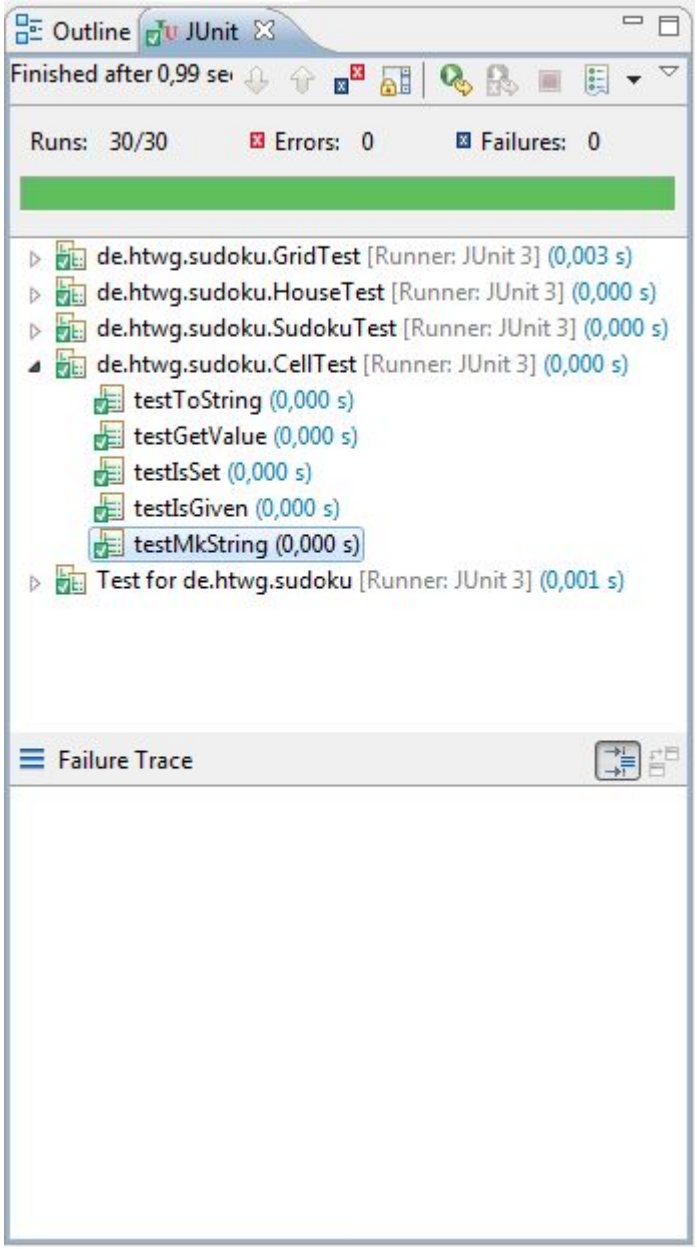
- Test Driven Development or Test First Development is one of the most prominent results of XP



JUnit

- Developed by Kent Beck and Erich Gamma
- To support the process of XP
- Translated to many languages





JUnit Java Code

```
import junit.framework.TestCase;
public class CellTest extends TestCase {
     Cell cell;
     public void setUp() {
        cell = new Cell(0,0);
     }
     public void testGetValue() {
        cell.setValue( 1 );
        assertEquals( 1, cell.getValue());
        cell.setValue(0);
        assertEquals( 0, cell.getValue());
     }
     public void testIsSet() {
        assertFalse( cell.isSet() );
        cell.setValue(1);
        assertTrue( cell.isSet() );
        cell.setValue(0);
        assertFalse( cell.isSet() );
     }
}
```

Pros and Cons of TDD

- Robert Martin
  - TDD is part of our professional discipline, like washing hands for a Doctor
- But with Junit, the test code can become very technical and hard to read and maintain
  - Some improved ideas in Ruby and in Scala

Behaviour Driven Development (BDD)

- Write a Specification in near-english code
- Then make that executable line after line
- Engages non-programmer into testing
- Technically it extends TDD
- But most importantly it creates a mind shift
  - Think of the tests as Specifications
  - And Tests are the best available Documentation

BDD in Scala

ScalaTest by Bill Venners is the most popular library for BDD in Scala

ScalaTest have several “Flavors” that provide a lot of flexibility for different people with different backgrounds or needs.

- FlatSpec, FunSpec, WordSpec, FreeSpec, PropSpec, FeatureSpec

You should start with WordSpec

To install it, add these two lines to your build.sbt file

```
libraryDependencies += "org.scalactic" %% "scalactic" % "3.2.14"
libraryDependencies += "org.scalatest" %% "scalatest" % "3.2.14" % "test"
```

WordSpec example

```
package de.htwg.se.tictactoe
import org.scalatest.wordspec.AnyWordSpec
import org.scalatest.matchers.should.Matchers._
class TicTacToeSpec extends AnyWordSpec {
 "TicTacToe" should {
   "have a bar as String of form '+---+---+---+'" in {
     bar() should be("+---+---+---+" + eol)
   }
   "have a scalable bar" in {
     bar(1, 1) should be("+-+" + eol)
     bar(1, 2) should be("+-+-+" + eol)
     bar(2, 1) should be("+--+" + eol)
   }
   "have cells as String of form '|   |   |   |'" in {
     cells() should be("|   |   |   |" + eol)
   }
   "have scalable cells" in {
     cells(1, 1) should be("| |" + eol)
     cells(1, 2) should be("| | |" + eol)
     cells(2, 1) should be("|  |" + eol)
   }
 }
```

Matchers

ScalaTest provides a domain specific language (DSL) for expressing assertions in tests using the word should. Just mix in Matchers, like this:

```
import org.scalatest._

class ExampleSpec extends WordSpec with Matchers {
    "A Cell" when {
      "not set to any value " should {
        val emptyCell = Cell(0)
        "have value 0" in {
          emptyCell.value should be(0)
        }
        "not be set" in {
          emptyCell.isSet should be(false)
        }
      }
```

Testing for equality

In JUnit the most frequent test is assertEquals.

```
assertEquals( 1, cell.getValue());
```

Or should it be …?

```
assertEquals( cell.getValue(), 1);
```

ScalaTest makes clear which is the result and what it is expected

```
result === expected // will provide both values on failure
expect(true){ some_Bool_Function }
result should equal (3)
```

And it offers alternatives with slightly different meaning

```
result should equal (3) // can customize equality
result should === (3)   // can customize equality and enforce type constraints
result should be (3)    // cannot customize equality, so fastest to compile
result shouldEqual 3    // can customize equality, no parentheses required
result shouldBe 3       // cannot customize equality, so fastest to compile, no
parentheses required
```

Matchers for Numbers

Odd and Even

```
num shouldBe odd
num should not be even
```

Less and Greater

```
one should be < 7
one should be > 0
one should be <= 7
one should be >= 0
```

Tolerance

```
sevenDotOh should equal (6.9 +- 0.2)
sevenDotOh should === (6.9 +- 0.2)
sevenDotOh should be (6.9 +- 0.2)
sevenDotOh shouldEqual 6.9 +- 0.2
sevenDotOh shouldBe 6.9 +- 0.2
```

Matchers for String

Matchers for String

```
string should startWith ("Hello")
string should endWith ("world")
string should include ("seven")
```

Matchers with RegEx

```
string should startWith regex "Hel*o"
string should endWith regex "wo.ld"
string should include regex "wo.ld"

string should fullyMatch regex """(-)?(\d+)(\.\d*)?"""

"abbccxxx" should startWith regex ("a(b*)(c*)" withGroups ("bb", "cc"))
"xxxabbcc" should endWith regex ("a(b*)(c*)" withGroups ("bb", "cc"))
"xxxabbccxxx" should include regex ("a(b*)(c*)" withGroups ("bb", "cc"))
"abbcc" should fullyMatch regex ("a(b*)(c*)" withGroups ("bb", "cc"))
```

Matchers for Collections

Emptiness

```
traversable shouldBe empty
javaMap should not be empty
```

Containment

```
traversable should contain ("five")

List(1, 2, 3, 4, 5) should contain oneOf (5, 7, 9)
Some(7) should contain oneOf (5, 7, 9)
"howdy" should contain oneOf ('a', 'b', 'c', 'd')

List(1, 2, 3, 4, 5) should contain noneOf (7, 8, 9)
Some(0) should contain noneOf (7, 8, 9)
"12345" should contain noneOf ('7', '8', '9')

(Array("Doe", "Ray", "Me") should contain oneOf ("X", "RAY", "BEAM")) (after being
lowerCased)


List(1, 2, 3) should contain atLeastOneOf (2, 3, 4)
Array(1, 2, 3) should contain atLeastOneOf (3, 4, 5)
"abc" should contain atLeastOneOf ('c', 'a', 't')
```

Matchers for Types

Classes and Types

```
result1 shouldBe a [Tiger]
result1 should not be an [Orangutan]
```

Generic types suffer erasure in the JVM

```
result shouldBe a [List[_]] // recommended
result shouldBe a [List[Fruit]] // discouraged
```

Matchers for Exceptions

Expect an Exception

```
an [IndexOutOfBoundsException] should be thrownBy s.charAt(-1)
```

Capture that Exception

```
val thrown = the [IndexOutOfBoundsException] thrownBy s.charAt(-1)
thrown.getMessage should equal ("String index out of range: -1")
```

In one step

```
the [ArithmeticException] thrownBy 1 / 0 should have message "/ by zero"
```

No Exception

```
noException should be thrownBy 0 / 1
```

Running Coverage Reports

For Scala 3.2 a code coverage a good code coverage tool is available. It is called scoverage.

For earlier Scala 3-versions, use jacoco

Add this to the file project/plugins.sbt

```
addSbtPlugin("org.scoverage" % "sbt-scoverage" % "x.x.x")
```

Then run

```
$ sbt clean coverage test

$ sbt coverageReport
```

Scoverage Report

All packages 75.28 %

com.sksamuel.scoverage.samples 75.28 %

SCoverage generated at Sun Dec 08 11:16:27 GMT 2013

Lines of code:	169	Files:	8	Classes:	8	Methods:	9
Lines per file	21.125	Packages:	1	Clases per package:	8.00	Methods per class:	1.13
Total statements:	89	Invoked statements:	67	Total branches:	12	Invoked branches:	9
Statement coverage:	75.28 %			Branch coverage:	75.00 %		

Class	Source file	Lines	Methods	Statements	Invoked	Coverage	Branches	Invoked	Coverage
ClientActor	ClientActor.scala	34	2	21	15	71.43 %	4	3	75.00 %
CreditEngine	CreditEngine.scala	13	1	7	7	100.00 %	2	2	100.00 %
InstrumentLoader	InstrumentLoader.scala	24	1	6	5	83.33 %	2	1	50.00 %
OrderEngine	OrderEngine.scala	22	1	14	14	100.00 %	0	0	0.00 %
Platform	Platform.scala	14	1	14	0	0.00 %	0	0	0.00 %
PriceEngine	PriceEngine.scala	31	1	10	10	100.00 %	2	2	100.00 %
QuoteCache	QuoteCache.scala	16	1	8	8	100.00 %	0	0	0.00 %
RandomQuoteGenerator	QuoteGenerator.scala	15	1	9	8	88.89 %	2	1	50.00 %

Task 3: Write Tests

- Take the Classes from your domain model
- Start developing them in Scala
- Use Behaviour Driven Development
  - Use ScalaTest
  - Write a Spec using WordSpec
  - Use the sbt test Test Runner
- Achieve 100% code coverage. And maintain it!